UNIT-V

Image Compression and Segmentation: Compression Models – Elements of Information Theory– Error Free Compression –Image Segmentation – Detection of Discontinuities – Edge Linking and Boundary Detection – Thresholding – Region Based Segmentation – Morphology.

Image Compression Models:

Three general techniques for reducing or compressing the amount of data required to represent an image. However, these techniques typically are combined to form practical image compression systems.

- A compression system consists of two distinct structural blocks: an encoder and a decoder'! An input image f(x, y) is fed into the encoder, which creates a set of symbols from the input data.
- After transmission over the channel, the encoded representation is fed to the decoder, where a reconstructed output image j(x, y) is generated. In general(x, y) may or may not be an exact replica of f(x, y). If it is, the system is error free or information preserving; if not, some level of distortion is present in the reconstructed image.
- It consists of two relatively in-dependent functions or sub blocks. The encoder is made up of a source encoder, which removes input redundancies, and a channel encoder, which increases the noise immunity of the source encoder's output.
- As would be expected, the de-coder includes a channel decoder followed by a source decoder. If the channel between the encoder and decoder is noise free (not prone to error), the channel encoder and decoder are omitted, and the general encoder and decoder become the source encoder and decoder, respectively.

Source Encoder and Decoder:

- The source encoder is responsible for reducing or eliminating any coding, interpixel, or psycho visual redundancies in the input image.
- The specific application and associated fidelity requirements dictate the best encoding approach to use in any given situation. Normally, the approach can be modeled by a series of three independent operations.

- Each operation is designed to reduce one of the three redundancies depicts the corresponding source decoder. In the first stage of the source encoding process, the mapper transforms the input data into a (usually nonvisual) format designed to reduce interpixel redundancies in the input image. This operation generally is reversible and may or may not reduce directly the amount of data required to represent the image.

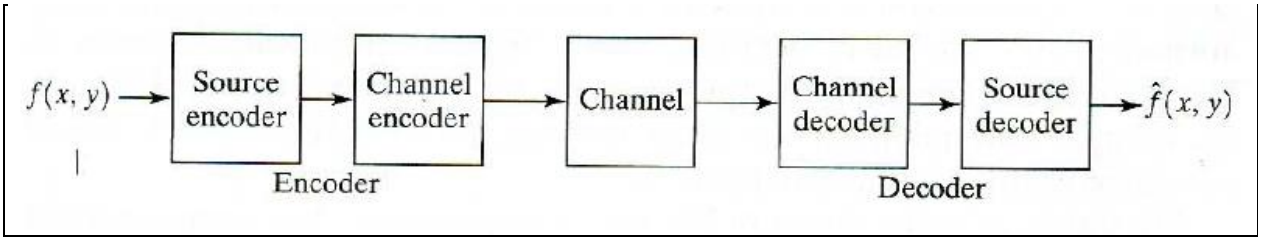


Fig :Source encoder model

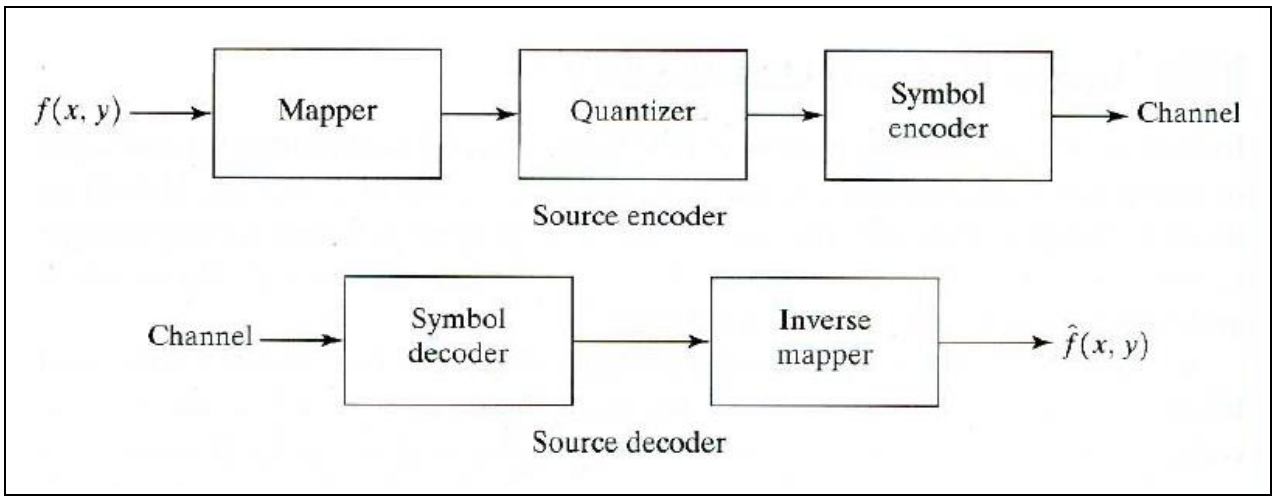


Fig: Source decoder model

- Run-length coding is an example of a mapping that directly results in data compression in this initial stage of the overall source en-coding process. The representation of an image by a set of transform coefficients is an example of the opposite case. Here, the mapper transforms the image into an array of coefficients, making its interpixel redundancies more accessible for compression in later stages of the encoding process.
- The second stage, or quantize, block reduces the accuracy of the mapper's output in accordance with some pre-established fidelity criterion. This stage reduces the psycho

visual redundancies of the input image. This operation is irreversible. Thus it must be omitted when error free compression is desired.

- In the third and final stage of the source encoding process, the symbol coder creates a fixed-or variable-length code to represent the quantizer output and maps the output in accordance with the code.
- The term symbol coder distinguishes this coding operation from the overall source encoding process. In most cases, a variable-length code is used to represent the mapped and quantized data set;it assigns the shortest code words to the most frequently occurring out- put values and thus reduces coding redundancy. The operation, of course, is reversible. Upon completion of the symbol coding step, the input image has been processed to remove each of the three redundancies.
- The source encoding process as three successive operations, but all three operations are not necessarily included in every compression system.
- The source decoder contains only two components: a symbol decoder and an inverse mapper. These blocks perform, in reverse order, the inverse operations of the source encoder's symbol encoder and mapper blocks because quantization results in irreversible information loss, an inverse quantizer block is not included in the general source decoder model .
- The channel encoder and decoder play an important role in the overall encoding-decoding process when the channel is noisy or prone to error.
- They are designed to reduce the impact of channel noise by inserting a con- trolled form of redundancy into the source encoded data. As the output of the source encoder contains little redundancy, it would be highly sensitive to trans- mission noise without the addition of this "controlled redundancy."
- One of the most useful channel encoding techniques was devised by R. W. Hamming (Hamming [1950]). It is based on appending enough bits to the data being encoded to ensure that some minimum number of bits must change be- tween valid code words. Hamming showed, for example, that if 3 bits of redundancy are added to a 4-bit word, so that the distance' between any two valid code words is 3, all single-bit errors can be detected and corrected. (By appending additional bits of redundancy, multiple-bit errors

can be detected and corrected.) The 7-bit Hamming (7, 4) code word h, h, ... h,h6h, associated with a 4-bit binary number $b_3b_2b_1b_0$is

$$h_1 = b_3 \oplus b_2 \oplus b_0 \qquad h_3 = b_3$$
$$h_2 = b_3 \oplus b_1 \oplus b_0 \qquad h_5 = b_2$$
$$h_4 = b_2 \oplus b_1 \oplus b_0 \qquad h_6 = b_1$$
$$h_7 = b_0$$

Where⊕ denotes the exclusive OR operation. Note that bits h" hz, and h4 are even- parity bits for the bit fields $b_3b_2b_0$, $b_3b_1b_0$, and $b_2b_1b_0$", respectively. (Recall that a string of binary bits has even parity if the number of bits with a value ofl is even.)

To decode a Hamming encoded result, the channel decoder must check the encoded value for odd parity over the bit fields in which even parity was previously established. A single-bit error is indicated by a nonzero parity word $c_4c_2c_1$, Where

$$c_1 = h_1 \oplus h_3 \oplus h_5 \oplus h_7$$
$$c_2 = h_2 \oplus h_3 \oplus h_6 \oplus h_7$$
$$c_4 = h_4 \oplus h_5 \oplus h_6 \oplus h_7$$

If a nonzero value is found, the decoder simply complements the code word bit position indicated by the parity word. The decoded binary value is then extracted from the corrected code word as $h_3h_5h_6h_7$.

Elements of Information Theory:

Several ways to reduce the amount of data used to represent an image.

Measuring Information:

- The fundamental premise of information theory is that the generation of in-formation can be modeled as a probabilistic process that can be measured in a manner that agrees with intuition.
- In accordance with this supposition, a random event E that occurs with probability p (E) is said to contain

$$I(E) = \log\frac{1}{P(E)} = -\log P(E)$$

Units of information. The quantity I(E) often is called the self-information of E. Generally speaking, the amount of self-information attributed to event E is inversely related to the probability of E. If P(E) = 1 (that is, the event always occurs), I(E) = 0 and no information is attributed to it.

The Information Channel:

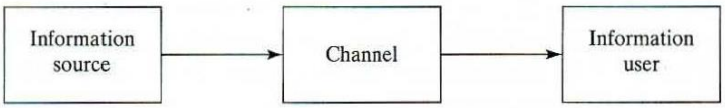
- When self-information is transferred between an information source and a user of the information, the source of information is said to be-connected to the user of information by an information channel.
- The information channel is the physical medium that links the source to the user. It may be a telephone line, an electromagnetic energy propagation path, or a wire in a digital computer. A simple mathematical model for a discrete information system. Here, the parameter of particular interest is the system's capacity, defined as its ability to transfer information.
- Generates a random sequence of symbols from a finite or countably infinite set of possible symbols. That is, the output of the source is a discrete random variable. The set of source symbols $\{a_1,a_2,..,a_J\}$ is referred to as the source alphabet A, and the elements of the set, denoted a_j, are called symbols or letters.
- The probability of the event that the source will produce symbol a_j is $P(a_j)$, and

$$\sum_{j=1}^{J} P(a_j) = 1.$$

A J X 1 vector $z = [P(a_1), p(a_2), \ldots, p(a_J)]^T$customarily is used to represent the set of all source symbol probabilities $\{P(a_1), p(a_2), \ldots, p(a_J)\}$ The finite ensemble (A, z) describes the information source completely.

The average information per source output, denoted H(z), is

$$H(\mathbf{z}) = -\sum_{j=1}^{J} P(a_j) \log P(a_j).$$



- This quantity is called the uncertainty or entropy of the source. It defines the average amount of information obtained by observing a single source output. As its magnitude increases, more uncertainty and thus more information is associated with the source.
- Having modeled the information source, we can develop the input-output characteristics of the information channel rather easily. Because we modeled the input to the channel in as a discrete random variable, the information transferred to the output of the channel is also a discrete random variable. Like the source random variable, it takes on values from a finite or countablyinfinite set of symbols $\{b_1, b_2, \ldots, b_K\}$ called the channel alphabet, B. The probability of the event that symbol b_k is presented to the information user is $P(b_k)$. The finite ensemble (B,v), where $v = [P(b_1), p(b_2), \ldots, P(b_K)]^T$, describes the channel output completely and thus the information received by the user.
- The probability $P(b_k)$ of a given channel output and the probability distribution of the source z are related by the expression

$$P(b_k) = \sum_{j=1}^{J} P(b_k \mid a_j) P(a_j)$$

Where$P(b_K \mid a_j)$ is the conditional probability that output symbol b_K is received, given that source symbol a_j was generated. If the conditional probabilities referenced in are arranged in a matrix K X J matrix **Q**, such that

$$\mathbf{Q} = \begin{bmatrix} P(b_1 \mid a_1) & P(b_1 \mid a_2) & \cdots & P(b_1 \mid a_J) \\ P(b_2 \mid a_1) & \cdot & \cdots & \cdot \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & \cdots & \cdot \\ \cdot & & & \\ P(b_K \mid a_1) & P(b_K \mid a_2) & \cdots & P(b_K \mid a_J) \end{bmatrix}$$

Then the probability distribution of the complete output alphabet can be computed from

$$\mathbf{v} = \mathbf{Qz}$$

Matrix **Q**, with elements $q_{kj} = P(b_K| a_j)$, is referred to as the forward channel *transition matrix* or by the abbreviated term channel matrix.

$$H(\mathbf{z}|b_k) = -\sum_{j=1}^{J} P(a_j|b_k)\log P(a_j|b_k)$$

Where$P(a_j | b_k)$,*)* is the probability that symbol $a_j$was transmitted by the source,given that the user received *bk*• The expected (average) value of this expression over all *bk* is

$$H(\mathbf{z}\|\mathbf{v}) = \sum_{k=1}^{K} H(\mathbf{z}|b_k)P(b_k),$$

which, after substitution of Eg. (8.3·7) for *H(***z**I *bk)* and some minor rearrangement,can be written as

$$H(\mathbf{z}|\mathbf{v}) = -\sum_{j=1}^{J}\sum_{k=1}^{K} P(a_j, b_k)\log P(a_j|b_k).$$

Here, $P(a_j, b_k)$ is the joint probability of $a_j$and b_k. That is, $p(a_j, b_k)$ is the probability that $a_j$is transmitted *and* $b_k$is received.

- The term H(**z**| **v**) is called the equivocation of z with respect to v. It represents the average information of one source symbol, assuming observation of the output symbol that resulted from its generation.
- Because H(**z**) is the average information of one source symbol, assuming no knowledge of the resulting output symbol, the difference between H(z) and H(z lv) is the average information received upon observing a single output symbol. This difference, denoted I(**z**, **v**) and called the mutual information of **z**and **v**, is

$$I(\mathbf{z}, \mathbf{v}) = H(\mathbf{z}) - H(\mathbf{z}|\mathbf{v})$$

for *H(**z**)* and H (**z**|**v**), and recalling that
$P(a_j) = P(a_j, b_1) + P(a_j' b_2,) + ... + P(a_j,b_k)$, yields

$$I(\mathbf{z}, \mathbf{v}) = \sum_{j=1}^{J}\sum_{k=1}^{K} P(a_j, b_k)\log\frac{P(a_j, b_k)}{P(a_j)P(b_k)}$$

This, after further manipulation, can be written as

$$I(\mathbf{z}, \mathbf{v}) = \sum_{j=1}^{J} \sum_{k=1}^{K} P(a_j) q_{kj} \log \frac{q_{kj}}{\sum_{i=1}^{J} P(a_i) q_{ki}}$$

Thus the average information received upon observing a single output of the informationchannel is a function of the input or source symbol probability vector**z**and channel matrix **Q**.

Error-Free Compression:

- In numerous applications error-free compression is the only acceptable means of data reduction. One such application is the archival of medical or business documents, where lossy compression usually is prohibited for legal reasons.
- Another is the processing of satellite imagery, where both the use and cost of collectingthe data makes any loss undesirable. Yet another is digital radiography, wherethe loss of information can compromise diagnostic accuracy. In these another cases, the need for error-free compression is motivated by the intendeduse or nature of the images under consideration.
- We focus on the principal error-free compression strategies currently in use. They normally provide compression ratios of 2 to 10. Moreover, they are equally applicable to both binary and gray-scale images.
- Error-free compression techniques generally are composed oftwo relatively independent operations:

  (1) Devising an alternative representationof the image in which its interpixel redundancies are reduced;

  (2) Coding the representation to eliminate coding redundancies. These steps correspondto the mapping and symbol coding operations of the source coding model discussedin connection

Variable-Length Coding:

- The simplest approach to error-free image compression is to reduce only codingredundancy. Coding redundancy normally is present in any natural binaryencoding of the gray levels in an image.Itcan be eliminated by coding the gray levels so is minimized.

- To do so requires construction of a variable-length code that assigns the shortest possiblecode words to the most probable gray levels. Here. We examine several optimaland near optimal techniques for constructing such a code.
- These techniques are formulated in the language of information theory. The sourcesymbolsmay be either thegray levels of an image or the output of a gray-level mappingoperation (pixel differences, run lengths, and so on).

Huffman coding:

- The most popular technique for removing coding redundancy is due to Huffman (Huffman [1952]).
- When coding the symbols of an information source individually, Huffman coding yields the smallest possible number of code symbols persource symbol.Interms of the noiseless coding theorem, theresulting code is optimal for a fixed value of n, subject to the constraint thatthe source symbols be coded one at a time.
- The first step in Huffman's approach is to create a series of source reductionsby ordering the probabilities of the symbols under consideration and combiningthe lowest probability symbols into a single symbol that replaces themin the next source reduction.
- At the far left, a hypotheticalset of source symbols and their probabilities are ordered from top to bottom in terms of decreasing probability values. To form the first source reduction, the bottom two probabilities, 0.06 and 0.04, are combined to form a "compound symbol" with probability 0.1.
- This compound symbol and its associated probabilityare placed in the first source reduction column so that the probabilities ofthe reduced source are also ordered from the most to the least probable. Thisprocess is then repeated until a reduced source with two symbols is reached.
- The second step in Huffman's procedure is to code each reduced source, startingwith the smallest source and working back to the original source. Theminimallength binary code for a two-symbol source, of course, is the symbols0and 1, thesesymbols are assigned to the two symbols onthe right (the assignment is arbitrary; reversing the order of the 0 and 1 wouldwork just as well). As the reduced source symbol with probability 0.6 was generatedby combining two symbols in the reduced source to its left, the 0 used tocode it is now assigned to *both* of these symbols, and a 0 and 1 are arbitrarily

Original source		Source reduction			
Symbol	Probability	1	2	3	4
a_2	0.4	0.4	0.4	0.4	0.6
a_6	0.3	0.3	0.3	0.3	0.4
a_1	0.1	0.1	0.2	0.3	
a_4	0.1	0.1	0.1		
a_3	0.06	0.1			
a_5	0.04				

Original source			Source reduction							
Sym.	Prob.	Code	1		2		3		4	
a_2	0.4	1	0.4	1	0.4	1	0.4	1	0.6	0
a_6	0.3	00	0.3	00	0.3	00	0.3	00	0.4	1
a_1	0.1	011	0.1	011	0.2	010	0.3	01		
a_4	0.1	0100	0.1	0100	0.1	011				
a_3	0.06	01010	0.1	0101						
a_5	0.04	01011								

Appended to each to distinguish them from each other. This operation is thenrepeated for each reduced source until the original source is reached. The finalcode appears at the far. The average length of this code is

$$L_{avg} = (0.4)(1) + (0.3)(2) + (0.1)(3) + (0.1)(4) + (0.06)(5) + (0.04)(5)$$
$$= 2.2 \text{ bits/symbol}$$

And the entropy of the source is 2.14 bits/ symbol. In accordance with the resulting Huffman code efficiency is 0.973.Huffman's procedure creates the optimal code for a set of symbols and probabilities subject to the constraint that the symbols be coded one at a time. After the code has been created, coding and/or decoding is accomplished in a simplelookup table manner.

Other near optimal variable length codes:

When a large number of symbols areto be coded, the construction of the optimalbinary Huffman code is a nontrivial task. For the general case of J source symbols,J - 2 source reductions must be performed and J - 2 codeassignments made. Thus construction of the optimal Huffmancode for an image with 256 gray levels requires 254 source reductions and 254code assignments. In view of thecomputational complexity of this task, sacrificingcoding efficiency for simplicity in code construction sometimes is necessary.

A truncated Huffman code is generated by Huffman coding only themost probable ψ symbols of the source,for some positiveintegerψless than$J$.Asecond, near optimal and variable-

lengthcode known as a B-code. It is close to optimal when the source symbol probabilitiesobey a power law of the form

$$P(a_j) = cj^{-\beta}$$

Forsome positive constant β and normalizing constant

$$c = 1/\sum_{j=0}^{J} j^{-\beta}$$

Forexample, the distribution of run lengths in a binary representation of a typical type written text document is nearly exponential.

Arithmetic coding:

- In arithmetic coding, which can be traced to the work of, a one-to-one correspondence between sourcesymbols and code words does not exist. Instead, an entire sequence of sourcesymbols (or message) is assigned a single arithmetic code word.
- The code word itself defines an interval of real numbers between 0 and 1. As the number of symbolsin the message increases, the interval used to represent it becomes smaller and the number of information units (say, bits) required to represent the interval becomes larger.
- Each symbol of the message reduces the size of the intervalin accordance with its probability of occurrence. Because the technique doesnot require, as does Huffman's approach, that each source symbol translate intoan integral number of code symbols (that is, that the symbols be coded one ata time), it achieves (but only in theory) the bound established by the noise less coding theorem
- Here, a five-symbol sequence or message, a1a2a3a3a4, from a four-symbol source is coded. At thestart of the coding process, the message is assumed to occupy the entire halfopeninterval [0,1).

LZW Coding:

- The technique, called Lempel-Ziv-Welch (LZW) coding, assigns fixed-length code words to variable length sequences of source symbols but requires no a priori knowledge of the probability of occurrence of the symbols to be encoded.
- **LZW** coding is conceptually very simple. At the onset of the Coding process, a code book or "dictionary" containing the source symbols tobe coded is constructed.
- For 8-bit monochrome images, the first 256 words of the dictionary are assigned to the gray values 0, 1, 2, ..., 255.
- As the encoder sequentially examines the image's pixels, gray-level sequences that are not in the dictionary are placed in algorithmically determined (e.g., the next unused) locations.If the first two pixels of the image are white, for instance, sequence"255-255" might be assigned to location 256, the address following the locations reserved for gray levels 0 through 255.
- The next time that two consecutive white pixels are encountered, code word 256, the address of the location containingsequence 255-255, is used to represent them. If a 9-bit, 512-word dictionary is employedin the coding process, the original (8 + 8) bits that were used to representthe two pixels are replaced by a single 9-bit code word.
- Cleary, the size of the dictionary is an imp9rtant system parameter. If it is too small, the detection of matching gray-level sequences will be less likely; if it is too large, the size ofthe code words will adversely affect compression performance.

Consider the following 4×4, 8-bit image of a vertical edge:

39	39	126	126
39	39	126	126
39	39	126	126
39	39	126	126

The steps involved in coding its 16 pixels. A 512-word dictionary with the following starting content is assumed:

Dictionary Location	Entry
0	0
1	1
⋮	⋮
255	255
256	—
⋮	⋮
511	—

Locations 256 through 511 are initially unused.

- A unique feature of the LZW coding just demonstrated is that the coding dictionary or code book is created while the data are being encoded; most practical applicationns require a strategy for handling dictionary overflow.
- A simple solution is to flush or reinitialize the dictionary when it becomes full and continue codingwith a new initializeddictionary.
- A more complex option is to monitor compression performance and flush the dictionary when it becomes poor orunacceptable. Alternately, the least used dictionary entries can be tracked andreplaced when necessary.

Bit-Plane Coding:

- Another effective technique for reducing an image's interpixel redundancies is to process the image's bit planes individually. The technique, called bit-planecoding, is based on the concept of decomposing a multilevel (monochrome orcolor) image into a series of binary images and compressing each binary imagevia one of several well-known binary compression methods.

Bit-plane decomposition:

The gray levels of an m-bit gray-scale image can be represented in the form of the base 2 polynomial

$$a_{m-1}2^{m-1} + a_{m-2}2^{m-2} + \cdots + a_1 2^1 + a_0 2^0.$$

Based on this property, a simple method of decomposing the image into a collection of binary images is to separate the m coefficients of the polynomial intom I -bit *bit planes.* As noted in Chapter 3, the zeroth-order bit plane is generated by collecting the *ao* bits of each pixel, while the *(m* - l)st-order bit plane contains the a_{m-1}, bits or coefficients. In general, each bit plane is numbered from0 to *m* - 1 and is constructed by setting its pixels equal to the values of the

appropriatebits or polynomial coefficients from each pixel in the original image.The inherent disadvantage of this approach is that small changes in gray levelcan have a significant impact on the complexity of the bit planes. If a pixel of intensity 127 (01111111) is adjacent to a pixel of intensity 128 (10000000), for instance, every bit plane will contain a corresponding 0 to 1 (or 1 to 0) transition.

For example, as the most significant bits of the two binary codes for 127 and 128 are different, bit plane 7 will contain a zero-valued pixel next to a pixel of value l, creating a 0 to 1 (or 1 to 0) transition at that point.

An alternative decomposition approach (which reduces the effect of small gray-level variations) is to first represent the image by an In-bit *Gray code.* The m-bit Gray code $g_{m-1} \ldots g_2 g_1 g_0$ that corresponds to the polynomial can be computed from

$$g_i = a_i \oplus a_{i+1} \qquad 0 \le i \le m-2$$
$$g_{m-1} = a_{m-1}.$$

Here, ⊕denotes the exclusive OR operation. This code has the unique property that successive code words differ in only one bit position. Thus, small changesin gray level are less likely to affect all *m* bit planes.

Constant area coding:

- A simple but effective method of compressing a binary image or bit plane is touse special code words to identify large areas of contiguous l's or 0's.
- In one such approach, called constant area coding (CAC), the image is divided into blocksof size p X q pixels, which are classified as all white, all black or mixed intensity. The most probable or frequently occurring category is then assigned the I-bit code word 0, and the other two categories are assigned the 2-bit codes 10 and 11.
- When predominantly white text documents are being compressed, a slightlysimpler approach is to code the solid white areas as 0 and all other blocks (including the solid black blocks) by a 1 followed by the bit pattern of the block. This approach, called white block skipping.

One-dimensional run-length coding:

An effective alternative to constant area coding is to represent each row of an image or bit plane by a sequence of lengths that describe successive runs ofblack and white pixels. This technique, referred to as run-length coding.

Two-dimensional run-length coding:

One-dimensional run-length coding concepts are easily extended to create a variety of 2-D coding procedures. One of the better known results is relative addresscoding (RAC), which is based on the principle of tracking the binary transitions that begin and end each black and white run.

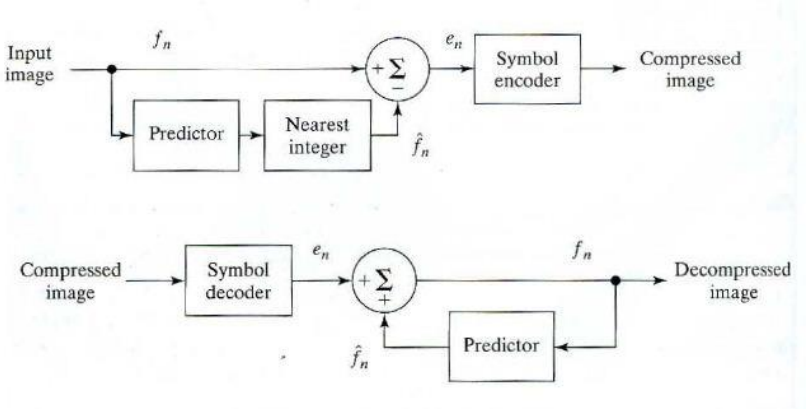
Contour tracing and coding:

Relative address coding is one approach for representing intensity transitions that make up the contours in a binary image. Another approach is to representeach contour by a set of boundary points or by a single boundary point and aset of directional. The latter sometimes is referred to as direct contour tracing.In this section, we describe yet another method, called predictive differential*quantizing* (PDQ), which demonstrates the essential characteristics of both approaches.It is a scan-line-oriented contour tracing procedure.

Lossless Predictive Coding:

- An error-free compression approach that does not require decomposition of an image into a collection of bit planes. The approach, commonly referred to as *lossless predictive coding*, isbased on eliminating theinterpixel redundancies of closely spaced pixels by extracting and coding *only*the new information in each pixel.
- The *new information* of a pixel is defined asthe difference between the actual and predicted value of that pixel. The basic components of a lossless predictive coding system.The system consists of an encoder and a decoder, each containing an identical *predictor*.
- As each successive pixel of the input image, denoted f_n, is introduced to the encoder, the predictor generates the anticipated value of that pixel based on some number of past inputs. The output of the predictor is thenrounded to the nearest integer, denoted f_n ,and used to form the difference or*prediction error*.

$$e_n = f_n - \hat{f}_n,$$



Which is coded using a variable-length code (by the symbol encode r) to generate the next element of the compressed data stream. The decoder of reconstructs *en* from the received variable-length code words and performs the inverse operation

$$f_n = e_n + \hat{f}_n.$$

Various local, global, and adaptive methods can be used to generate f_n. In most cases, however. The prediction is formed by a linear combination of m previous pixels. That is,

$$\hat{f}_n = \text{round}\left[\sum_{i=1}^{m} \alpha_i f_{n-i}\right]$$

Where *m* is the order of the linear predictor, round is a function used to denote the rounding or nearest integer operation, and the α_i for i = 1, 2, ... ,*m* are prediction coefficients.. In I-D linear predictive coding, for example

$$\hat{f}_n(x, y) = \text{round}\left[\sum_{i=1}^{m} \alpha_i f(x, y - i)\right]$$

A similar comment applies to the higher-dimensional cases. Consider encoding the monochrome image of using the simple first-order linear predictor

$$\hat{f}(x, y) = \text{round}[\alpha f(x, y - 1)].$$

- A predictor of this general form commonly is called a previous pixel predictor, and the corresponding predictive coding procedure is referred to as differential coding or previous pixel *coding.*
- The preceding example emphasizes that the amount of compression achieved in loss less predictive coding is related directly to the entropy reduction that results from mapping the input image into the prediction error sequence.
- Because a great deal of inter pixel redundancy is removed by the prediction and differencing process, the probability density function of the prediction error is, in general, highly peaked at zero and characterized by a relatively small (in comparison to theinput gray-level distribution) variance. In fact, the density function of the predictionerror often is modeled by the zero mean uncorrelated Laplacian pdf

$$p_e(e) = \frac{1}{\sqrt{2}\sigma_e} e^{\frac{-\sqrt{2}|e|}{\sigma_e}}$$

Where (f f' is the standard deviation of *e)*

Detection of Discontinuities:

Several techniques for detecting the three basic typesof gray-level discontinuities in a digital image: points, lines, and edges. The mostcommon way to look for discontinuities is to run amask through the image in themanner described in Section 3.5. For the 3 X 3 mask shown in Fig. 10.1, this procedureinvolves computing the sum of products of the coefficients with the gray

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

Levels contained in the region encompassed by the mask. That is, with reference to the response of the mask at any point in the image is given by

$$R = w_1 z_1 + w_2 z_2 + \cdots + w_9 z_9 = \sum_{i=1}^{9} w_i z_i$$

Where *Zi*is the gray level of the pixel associated with mask coefficient *Wi* .As usual, the response of the mask is defined with respect to its center location. Thedetails for implementing mask operations.

Point Detection:

The detection of isolated points in an image is straightforward in principle. Using the mask shown We say that a point has been detected atthe location on which the mask is centered if

$$|R| \geq T$$

Where*T* is a nonnegative threshold and *R* is given .Basically this formulation measures the weighted differences between the center pointand its neighbors. The idea is that an *isolated* point (a point whose gray level issignificantly different from its background and which is located in a homogeneousor nearly homogeneous area) will be quite different from its surroundings,and thus be easily detectable by this type of mask. Onlydifferences that are considered of interest are those

−1	−1	−1
−1	8	−1
−1	−1	−1

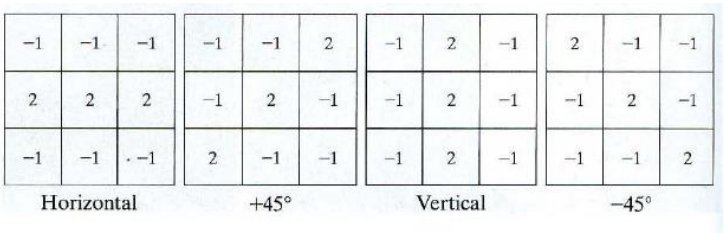
Largeenough (as determined by *T)* to be considered isolated points.

Line Detection:

- The next level of complexity is line detection. Consider the masks. If the first mask were moved around an image, it would respond more strongly to lines (one pixel thick) oriented horizontally.
- With a constant background, the maximum response would result when the line passed through the middle row of the mask. This is easily verified by sketching a simple array of

l's with a line of a different gray level (say, 5's) running horizontally through the array. A similar experiment would reveal that the second mask .responds best to lines oriented at +45°; the third mask to vertical lines; and the fourth mask to lines inthe -45° direction. These directions can be established also by noting that the preferred direction of each mask is weighted with a larger coefficient (i.e., 2) than other possible directions.

- Note that the coefficients in each mask sum to zero, indicating a zero response from the masks in areas of constant gray level.
- Let *R" R" R3* ,and *R4* denote the responses of the masks.If, at a certain point in the image,$|R_i| > |R_j|$, for all $j \neq i$, that point is said to be more likely associated with a linein the direction of mask i. For example, if at a point in the image, $|R_1| > |R_j|$, for



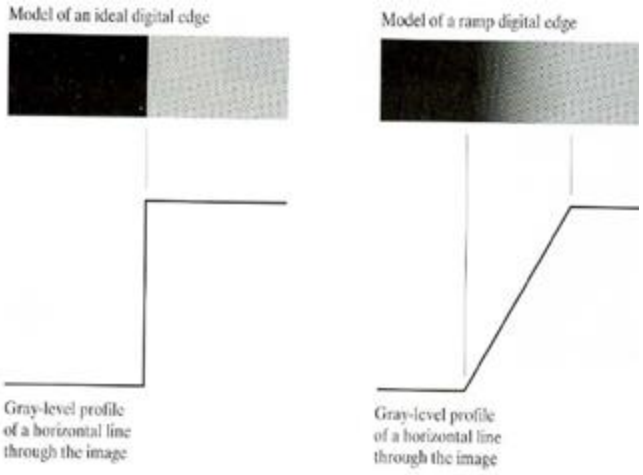
j = 2, 3, 4. Thatparticular point is said to be more likely associated with a horizontal line.

Edge Detection:

Although point and line detection certainly are important in any discussion on segmentation, edge detection is by far the most common approach for detecting meaningful discontinuities in gray level.

Basic formulation:

An edge is a set of connectedpixels that lie on the boundary between two regions. However, we alreadywent through some length .We start by modeling an edge intuitively. This will lead us to a formalism in which "meaningful" transitions in gray levels can be measured. Intuitively, an ideal edge has the properties of the model.



Gradient operators:

First-order derivatives of a digital image are based on various approximations of the 2-D gradient. The gradient of an image *f(x, y)* at location *(x , y)*is defined as the *vector*

$$\nabla \mathbf{f} = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \dfrac{\partial f}{\partial x} \\ \dfrac{\partial f}{\partial y} \end{bmatrix}.$$

It is well known from vector analysis that the gradient vector points in the direction of maximum rate of change of **f** at coordinates *(x, y)*.An important quantity in edge detection is the magnitude of this vector,denoted V *f*, where

$$\nabla f = \text{mag}(\nabla \mathbf{f}) = \left[G_x^2 + G_y^2\right]^{1/2}.$$

This quantity gives the maximum rate of increase of *f(x, y)* per unit distance in the direction of ∇f .It is a common (although not strictly correct) practice torefer to ∇f also as the *gradient.*The *direction* of the gradient vector also is an important quantity. Let *a(x ,y)* represent the direction angle of the vector ∇f at *(x , y).* Then, from vector analysis,

$$\alpha(x, y) = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

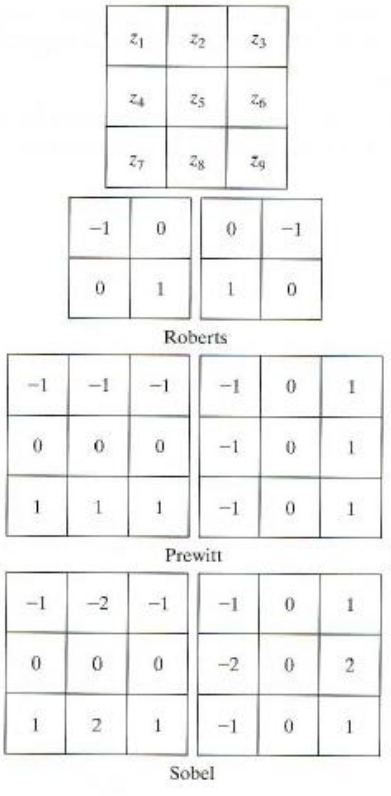
One of the simplest ways to implement a first-order partial derivative at point $Z_5$is to use the following Roberts cross-gradient operators

$$G_x = (z_9 - z_5)$$

$$G_y = (z_8 - z_6).$$

Masks of size 2 X 2 are awkward to implement because they do not have a clear center. An approach using masks of size 3 X 3 is given by

$$G_x = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$



The difference between the first and third rows of the 3 X 3 image region approximates the derivative in the x-direction, and the difference between the third and first columns approximates the derivative in the y-direction.The masks called the Prewitt operators, canbe usedto implement these two' equations.

A slight variation of these two equations uses a weight of 2 in the center coefficient:

$$G_x = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

$$G_y = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7).$$

An approach used frequently is to approximate the gradientby absolute values:

$$\nabla f \approx |G_x| + |G_y|.$$

The Laplacian

The Laplacian ofa 2-D function *f(x, y)* is a second-order derivative defined as

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}.$$

Digital approximations to the Laplacian were introduced in Section 3.7.2. For a 3 X 3 region, one of the two forms encountered most frequently in practice is

$$\nabla^2 f = 4z_5 - (z_2 + z_4 + z_6 + z_8)$$

0	−1	0	−1	−1	−1
−1	4	−1	−1	8	−1
0	−1	0	−1	−1	−1

Wherethe z's are defined .A digital approximation including the diagonal neighbors is given by

$$\nabla^2 f = 8z5 - (z, + z, + Z3 + z, + <6 + <7 + Z8 + z,).$$

In the first category, the Laplacian is combined with smoothing as a precursorto finding edges via zero-crossings. Consider the function

$$h(r) = -e^{-\frac{r^2}{2\sigma^2}}$$

Where$r' = x' + y'$ and *(J* is the standard deviation. Convolving this function with an image blurs the image, with the degree of blurring being determined by thevalue of *(J.* The Laplacian of *h* (the second derivative of *h* with respect to *r)* is

$$\nabla^2 h(r) = -\left[\frac{r^2 - \sigma^2}{\sigma^4}\right] e^{-\frac{r^2}{2\sigma^2}}.$$

Edge Linking and Boundary Detection:

This set of pixels seldom characterizes an edge completely because of noise, breaks in the edge from non-uniform illumination, and other effects that introduce spurious intensity discontinuities. Thus edge detection algorithms typically are followed by linking procedures to assemble edgepixels into meaningful edges.

Local Processing:

- One of the simplest approaches for linking edge points is to analyze the characteristics of pixels in a small neighborhood (say, 3 X 3 or 5 X 5) about every point *(x, y)* in an image that has been labeled an edge point by one of the techniques.
- All points that is similar according to a set ofPredefinedcriteria are linked, forming an edge of pixels that share those criteria.
- The two principal properties used for establishing similarity of edge pixels in this kind of analysis are

  (1) The strength of the response of the gradient operatorused to produce the edge pixel; (2) the direction of the gradient vector. Thefirst property is given by the value of ∇f , as defined Thus an edge pixel with coordinates *(xo, yo)* in a predefined neighborhood of*(x, y)* , is similar in magnitude to the pixel at *(x, y)* if

$$|\nabla f(x, y) - \nabla f(x_0, y_0)| \leq E$$

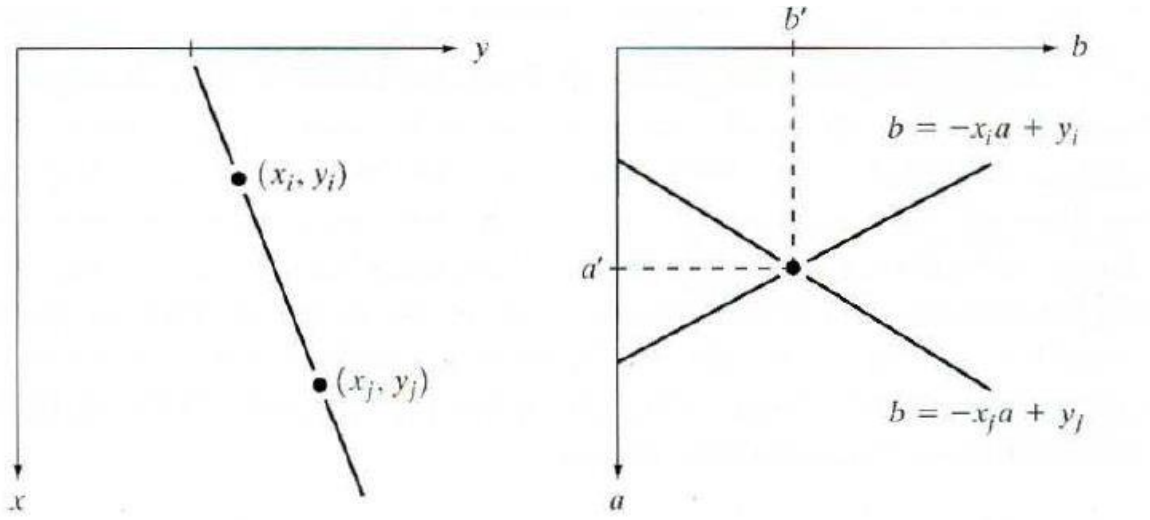
Where *E* is a non negative Threshold. The direction (angle) of the gradient vectoris given. An edge pixel at (xo, *yo)* in the predefined neighborhood of *(x, y)* has an angle similar to the pixel at *(x. y)* if

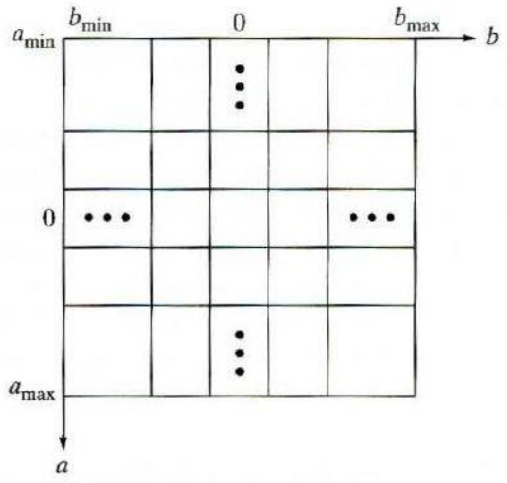
$$|\alpha(x, y) - \alpha(x_0, y_0)| < A$$

Where A is a non negative angle threshold. The direction of the edge at *(x, y)* is *perpendicular* to the direction of the gradient vectorat that point.

Global Processing via the Hough Transform

- Points are linked by determining first if they lie on a curve of specified shape. Unlike the local analysis method we now consider global relationships between pixels.
- One possible solution is to first find all lines determined by every pair of points and then find all subsets of points that areclose to particular lines.
- The problem with this procedure is that it involves finding$n\ (n - 1)/2$ - n' lines and then performing $(n)\ (n(n - 1))/2 - n3$ comparisons of every point to all lines.
- This approach is computationally prohibitive in all but the most trivial applications.Consider a point (x_i, y_i) and the general equation of a straightline in slope-intercept form, $y_i = ax_i + b$. In finitely many lines pass through (x_i, y_i), but they all satisfy the equation $yi = ax_i + b$ for varying values of a and b. However, writing this equation as $b = -x_i a + y_i$and considering the ab-plane (also called *parameter space)* yields the equation of a *single* line for a fixed pair$(x_i.\ y_i)$· Furthermorea second point $(xj.\ y_i)$ also has a line in parameter space associatedwith it, and this line intersects the line associated with (xj,y_i) at (a', b'),where a' is the slope and b' the intercept of the line containing both $(xj.\ y_i)$ and$(xj.\ y_i)$ in the xy-plane. In fact, all points contained on this line have lines in parameter space thatintersect at (a', b').



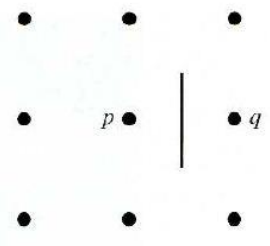


The computational attractiveness of the Hough transform arises from subdividing the parameter space into so-called *accumulator cells*, as illustrated,where (a_{max},a_{min}) and (b_{max},b_{min}) are the expected ranges of slopeand intercept values. The cell at coordinates *(i, j),* with accumulator value *A(i, j),*corresponds to the square associated with parameter space coordinate (a_i,b_j).Initially, these cells are set to zero. A problem with using the equation $Y = ax + b$ to represent a line is thatthe slope approaches infinity as the line approaches the vertical. One way around this difficulty is to use the normal representation of a line:

$$x\cos\theta + y\sin\theta = p.$$

Global Processing via Graph-Theoretic Techniques

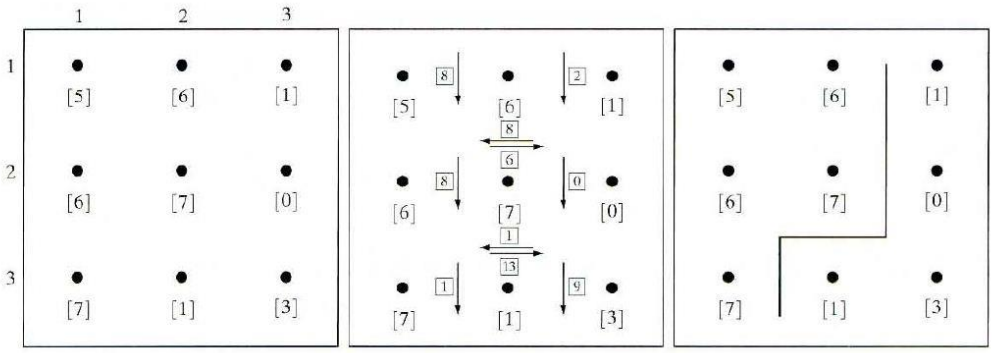
A global approach for edge detection and linking based on representing edge segments in the form of a graph and searching the graphfor low-cost paths that correspond to significant edges. This representation providesa rugged approach that performs well in the presence of noise. As might be expected, the procedure is considerably more complicatedand requires moreprocessing time than



We begin the development with some basic definitions. A *graph* G= *(N , U)* is a finite, nonempty set of nodes *N*, together with a set *U* of unordered pairs of distinct elements of *N*. Each pair (n_i, n_J) *ofU* is called an *arc*. *A* graph in which the arcs are directed is called a *directed graph*

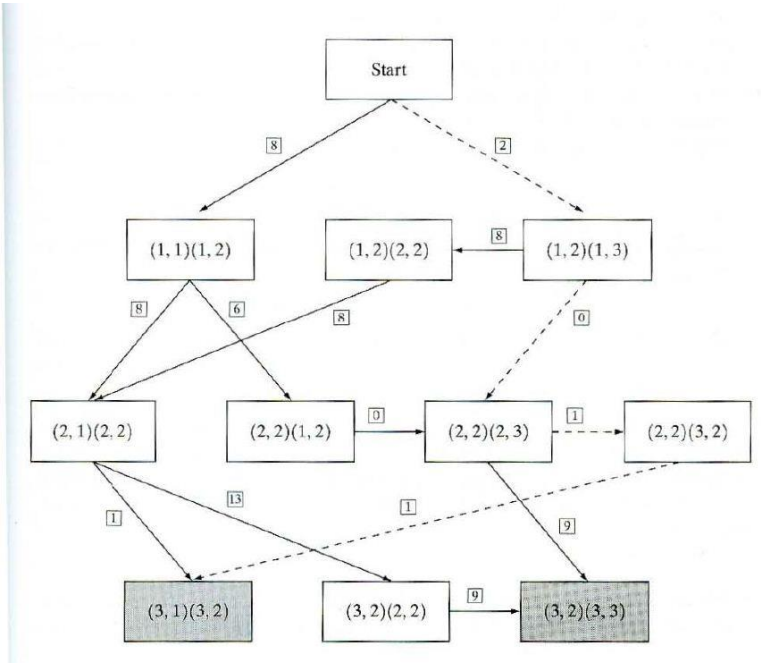
$$c = \sum_{i=2}^{k} c(n_{i-1}, n_i).$$

The following discussion is simplified if we define an edge elementas the boundary between two pixels *p* and *q*, such that *p* and *q* are 4·neighbors, Edge elements are identified by the xy-coordinates of points *p* and *q*. In other words, the edge element in Fig. 10.22 is defined by the pairs$(x_p, Y_p)(X_q, Y_q)$. Consistent with the definition given in Section 10.1.3, an *edge*is a sequence of connected edge elements.



- Where H is the highest gray-level value in the image (7 in this case), and *f(p)*and *f(q)* are the gray-level values of *p* and *q*, respectively. By convention, thepoint *p* is on the right-hand side of the direction of travel along edge elements.For example, the edge segment (1,2)(2, 2) is between points (1,2) and (2,2) If the direction of travel is to the right, then *p* is the point with coordinates (2, 2) and *q* is point with coordinates (1 , 2); therefore,*c(p, q)* = 7 - [7 - 6] = 6.
- This cost is shown in the box below the edge segment.If,on the other hand, we are traveling to the *left* between the same twopoints, then *p* is point (1, 2) and *q* is (2,2). In this case the cost is 8, as shownabove the edge segment.

- To simplify the discussion, we assumethat edges start in the top row and terminate in the last row, so that thefirst element of an edge can be only between points (1 ,1), (1, 2) or (1 , 2),(1, 3). Similarly, the last edge element has to be between points (3, 1), (3,2)or (3,2), (3,3). Keep in mind that *p* and *q* are 4-neighbors, as noted earlier.



Step 1: Mark the start node OPEN and set g(s) = 0.

Step 2: If no node is OPEN exit with failure; otherwise, continue.

Step 3: Mark CLOSED the OPEN node n whose estimate r(n) computed from Eg. (10.2·7) is smallest. (Ties for minimum r values are resolved arbitrarily, but always in favor of a goal node.)

Step 4: If n is a goal node, exit with the solution path obtained by tracingback through the pointers; otherwise, continue.

Step 5: Expand node n, generating all of its successors. (If there are no successors go to step 2.)

Step 6: If a successor n; is not marked, setmark it OPEN, and direct pointers from it back to n.

$$r(n_i) = g(n) + c(n, n_i),$$

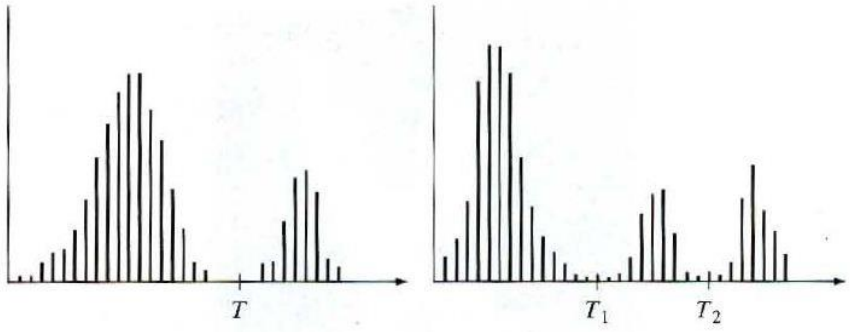
Step 7: If a successor 11; is marked CLOSED or OPEN, update its value byLetting

$$g'(n_i) = \min[g(n_i), g(n) + c(n, n_i)].$$

Mark OPEN those CLOSED successors whose g' values were thus loweredand redirect to n the pointers from all nodes whose g' values were lowered.Go to step 2.

Thresholding:

- Thresholding in a moreformal way and extend it to techniques that are considerably more general Suppose that the gray- level histogram corresponds to an image ,f(x, y), composed of light objects on a dark background, in such a waythat object and background pixels have gray levels grouped into two dominant modes.
- One obvious way to extract the objects from the background is 10 selecta threshold thatseparates these modes. Then any point (x, y) for whichf(x, y) >T is called an object point; otherwise, the point is called a backgroundpoint. This is the type of thresholding



Based on the preceding discussion, thresholding may be viewed as an operation that involves tests against a function Tof the form

$$T = T[x, y, p(x, y), f(x, y)]$$

where J(x, y) is the gray level of point (x, y) and p(x, y) denotes some local property of this point- for example, the average gray level of a neighborhoodcentered on (x, y).Athresholded image g(x, y) is defined as

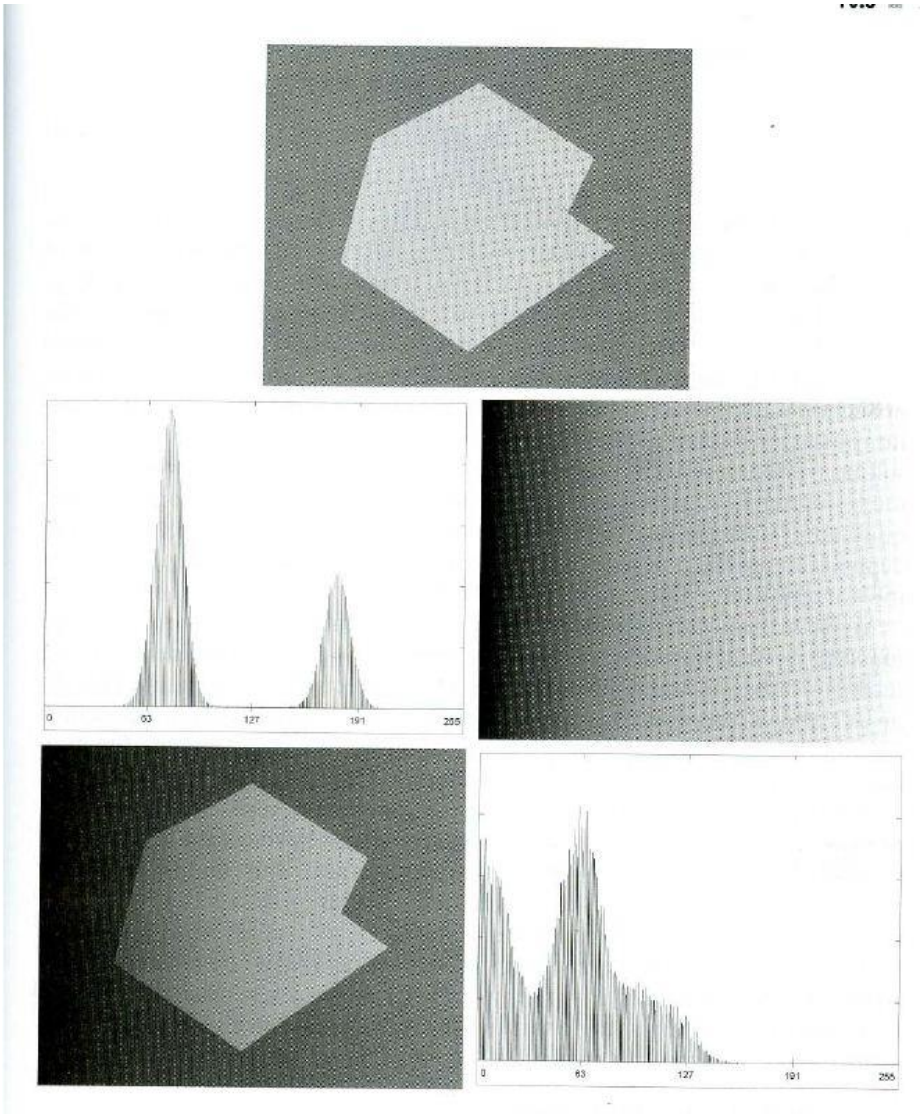
$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{if } f(x, y) \leq T. \end{cases}$$

Thus, pixels labeled 1 (or any other convenient gray level) correspond to objects, whereaspixels labeled 0 (or any other gray level not assigned to Objects) correspondto the background.When T depends only on J(x,y) (that is, only on gray-level values) thethreshold is called global. If T

depends on both *J(x, y)* and *p(x, y),*the thresholdis called *local.* If, in addition, *T* depends on the spatial coordinates *x* and *y,*the threshold is called *dynamic* or *adaptive.*

The Role of Illumination

A simple model in which an image *J(x ,y)* is formed as the product of a reflectance component *r(x, y)* and an illumination component *i(x, y).* The purpose of this section is to use this model to discuss briefly theeffect of illumination on thresholding, especially on global thresholding.



$$f(x, y) = i(x, y)r(x, y).$$

Taking the natural logarithm of this equation yields a sum:

$$\begin{aligned} z(x, y) &= \ln f(x, y) \\ &= \ln i(x, y) + \ln r(x, y) \\ &= i'(x, y) + r'(x, y). \end{aligned}$$

Basic Global Thresholding:

- The simplest of all thresholding techniques is to partition the image histogram by using a single globalthreshold, *T,* as illustrated.
- Segmentation is then accomplished by scanning the image pixel by pixel and labeling each pixel as object or background, depending on whether the gray level of that pixel is greater or less thanthe value of *T.* The following algorithm can be used to obtain *T* automatically:

1.Select an initial estimate for *T.*

2. Segment the image using *T.* This will produce two groups of pixels: G, consisting of all pixels with gray level values >*T* and G2 consisting of pixels with values ~ *T.*

3. Compute the average gray level values 1-' ,and 1-'2 for the pixels in regions $G_1$and G_2.

4. Compute a new threshold value:

$$T = \frac{1}{2}(\mu_1 + \mu_2).$$

5. Repeat steps 2 through 4 until the difference in *T* in successive iterations is smaller than a predefined parameter *To.*

Basic Adaptive Thresholding

All subimages containing boundaries had variances in excess of 100. Each subimage with variance greater than 100 wassegmented with a threshold computed for that subimage using the algorithm discussedin the previous section. The initial value for *T* in each case was selected as the point midway between the minimum and maximum gray levels in the subimage. All subimages with variance less than 100 were treated as one compositeimage, which was segmented using a single threshold estimated using the same algorithm.

Optimal Global and Adaptive Thresholding

- The method is applied to a problem that requires sol1ltion of several important issues found frequently in the practical application of thresholding.Suppose that an image contains only two principal gray- level regions. Let *z* denote gray-level values.
- We can view these values as random quantities, andtheir histogram may be considered an estimate of their probability density function(PDF), *p(z).* This overall density function is the sum or mixture of two densities, one for the light and the other for the dark regions in the image.
- Furthermore, the mixture parameters are proportional to the relative areas of the dark and light regions. If the form of the densities is known or assumed, itis possible to determine an optimal threshold (in terms of minimum error) forsegmenting the image into the two distinct regions. Assume that the largerof the two PDFs corresponds to the background levels while the smaller onedescribes the gray levels of objects in the image. The mixture probability densityfunction describing the overall gray-level variation in the image is

$$p(z) = P_1 p_1(z) + P_2 p_2(z).$$

Here, P_1 and P_2 arethe probabilities of occurrence of the two classes of pixels; that is, P_1 is the probability (a number) that a random pixel with value z is anobject pixel. Similarly, $P_2$is the probability that the pixel is a background pixel.We are assuming that any given pixel belongs either to an object or to the background,so that

$$P_1 + P_2 = 1.$$

Thus, the probability of *erroneously* classifying a background point as an object point is

$$E_1(T) = \int_{-\infty}^{T} p_2(z)\, dz.$$

This is the area under the curve of $p,(z)$ to the left of the threshold. Similarly,the probability of erroneously classifying an object point as background is

$$E_2(T) = \int_T^{\infty} p_1(z)\, dz,$$

Whichis the area under the curve of $p,(z)$ to the right of T. 111en the overall probability of error is

$$E(T) = P_2 E_1(T) + P_1 E_2(T).$$

Note how the quantities $E_1$and $E_2$are weighted (given importance) by the probability of occurrence of object or background pixels. Note also that the sub-scripts are opposites

To find the threshold value for which this error is minimal requires differentiating $E(T)$ with respect to T (using Leibniz's rule) and equating the resultto 0. The result is

$$P_1 p_1(T) = P_2 p_2(T).$$

One of the principal densities used in this manner is the Gaussian density, which is completely characterized by two parameters:the mean and the variance. In this case

$$p(z) = \frac{P_1}{\sqrt{2\pi}\sigma_1} e^{-\frac{(z-\mu_1)^2}{2\sigma_1^2}} + \frac{P_2}{\sqrt{2\pi}\sigma_2} e^{-\frac{(z-\mu_2)^2}{2\sigma_2^2}}$$

Where$\mu_1$and $crT$are the mean and variance of the Gaussian density of one classof pixels (say, objects) and μ_2. And O'jare the mean and variance of the other class. Using this equation in the general solution

$$AT^2 + BT + C = 0$$

$$\begin{aligned} A &= \sigma_1^2 - \sigma_2^2 \\ B &= 2(\mu_1\sigma_2^2 - \mu_2\sigma_1^2) \\ C &= \sigma_1^2\mu_2^2 - \sigma_2^2\mu_1^2 + 2\sigma_1^2 2\sigma_2^2 \ln(\sigma_2 P_1/\sigma_1 P_2). \end{aligned}$$

Since a quadratic equation has two possible solutions, two threshold values may be required to obtain the optimal solution. if the variances are equal, *(T2 = O"T = uL*a single threshold is sufficient:

$$T = \frac{\mu_1 + \mu_2}{2} + \frac{\sigma^2}{\mu_1 - \mu_2} \ln\left(\frac{P_2}{P_1}\right).$$

For example, the mean square error between the (continues) mixture density *p(z)* and the (discrete) image histogram *h(Zi)* is

$$e_{\text{ms}} = \frac{1}{n} \sum_{i=1}^{n} [p(z_i) - h(z_i)]^2$$

The chances of selecting a "good" threshold are enhanced considerably if the histogram peaks are tall, narrow, symmetric, and separated by deep valleys. One approach for improving the shape of histograms is to consider only those pixels that lie on or near the edges between objects and the background. Thresholds Based on Several Variables

A sensor can make available more than one variable to characterize each pixel in an image, and thus allow multispectral thresholding.

Region-Based Segmentation

- This problem by finding boundaries between regions based on discontinuities in gray levels, whereas segmentation was accomplished via thresholds based on the distribution of pixel properties, such as gray-level values or color. In this section we discuss segmentation techniques that are based on finding the regions directly. That partitions *R* into *n* sub regions, $R_1R_2 \ldots R_n$ such that

(a) $\bigcup_{i=1}^{n} R_i = R.$
(b) R_i is a connected region, $i = 1, 2, \ldots, n.$
(c) $R_i \cap R_j = \varnothing$ for all i and $j, i \neq j.$
(d) $P(R_i)$ = TRUE for $i = 1, 2, \ldots, n.$
(e) $P(R_i \cup R_j)$ = FALSE for $i \neq j.$

Here, *p(Ri)* is a logical predicate defined over the points in set *Ri* and 0 is the null set.

Condition (a) :indicates that the segmentation must be complete; that is, every pixel must be in a region.

Condition (b): requires that points in a region must be connected in some predefined sense

Condition (c) : indicates that the regions must be disjoint.

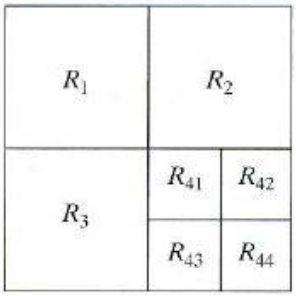
Condition(d):deals with the properties that must be satisfied by the pixels in a segmented region-for example *p(*RJ = TRUE if a li pixels in *R,* have the same gray level. finally,

Condition(e): Indicates that regions *R,* and *Rj* are different in the sense of predicate *P.*

Region Growing

- *Region growing* is a procedure that group's pixels or sub regions into larger regions based on predefined criteria.1l1e basic approach is to start with a set of "seed" points and from these grow regions by appending to each seed those neighboring pixels that have properties similar to the seed (such as specific ranges of gray level or color).
- Selecting a set of one or more starting points often can be based on the nature of the problem, as. When a priori information is not available, the procedure is to compute at every pixel the same set of properties that ultimately will be used to assign pixels to regions during the growing process.
- If the result of these computations shows clusters of values, the pixels whose properties place them near the centroid of these clusters can be used as seeds. This location of similarity criteria depends not only on the problem under consideration, but also on the type of image data available.
- For example, the analysis of land-use satellite imagery depends heavily on the use of color. This problem would be significantly more difficult, or even impossible, visualize a random arrangement of pixels with only three distinct gray-level values. Grouping pixels with the same gray level to form a "region" without paying attention to connectivity would yield a segmentation result that is meaningless in the context
- Another problem in region growing is the formulation of a stopping rule. Basically, growing a region should stop when no more pixels satisfy the criteria for inclusion in that region. Criteria such as gray level, texture, and color, are local in nature and do not take into account the "history" of region growth.
- An alternative is to subdivide an image initially into a set of arbitrary, disjointed regions and then merge and or split the regions in an attempt to satisfy the conditions stated

Let *R* represent the entire image region and select a predicate *P*. One approach for segmenting *R* is to subdivide it successively into smaller and smaller quadrant regions so that, for any region *Ri* ,*p(Ri)* = TRUE. We start with the entire region. If *P(R)* = FALSE, we divide the image into quadrants. If *P* is FALSE for any quadrant, we subdivide that quadrant into sub quadrants, and soon. This particular splitting technique has a convenient representation in the form of a so-called *quad tree*(that is, a tree in which nodes have exactly four descendants).



1. Split into [our disjoint quadrants any region *R;* for which *p(R;)* = FALSE.
2. Merge any adjacent regions R_j and *R,* for which $P(R_j \cup R,)$ = TRUE.
3. Stop when no further merging or splitting is possible.

Segmentation by Morphological Watersheds:

- Segmentation based on three principal concepts :(a) detection of discontinuities, (b) thresholding, and (c) region processing. Each of these approaches was found to have advantages (for example, speed in the case of global thresholding) and disadvantages (for example, the need for post processing, such as edge linking, in methods based on detecting discontinuities in gray levels).
- The concept of watersheds is based on visualizing an image in three dimensions: two spatial coordinates versus gray levels.
- In such a " topographic" interpretation, we consider three types of points:

  (a) Points belonging to a regional minimum;

  (b) Points at which a drop of water, if placed at the location of any of those points, would fall with certainty to a single minimum;

(c) Points at which water would be equally likely to rail to more than one such minimum. For a particular regional minimum, the set of points satisfying condition (b) is called the *catchment basin* or *watershed* of that minimum. The point's satisfying condition(c) Form crest lines on the topographic surface and are termed *divide lines* or *watershed lines.*

- The basic idea is simple: Suppose that a hole is punched in each regional minimum and that the entire topography is flooded from below by letting water rise through the holes at a uniform rate.
- The flooding will eventually reach a stage when only the tops of the dams are visible above the water line. These dam boundaries correspond to the divide lines of the watersheds. Therefore, they are the (continuous) boundaries extracted by a watershed segmentation algorithm.
- Three-dimensional representation is of interest. In order to prevent the rising water from spilling out through the edges of the structure, we imagine the perimeter of the entire topography (image) being enclosed by dams of height greater than the highest possible mountain, whose value is determined by the highest possible gray-level value in the input image.
- Suppose that a hole is punched in each regional minimum and that the entire topography is flooded by letting water rise through the holes at a uniform rate. Figure 1O.44(c) shows the first stage of flooding, where the "water," shown in light gray, has covered only areas that correspond to the very dark background in the image.
- That the water now has risen into the first and second catchment basins, respectively. As the water continues to rise, it will eventually overflow from one catchment basin into another. The first indication Here, water from the left basin actually overflowed into the basin on the right and a short "dam " (consisting of single pixels) was built to prevent water from merging at that level of flooding (the details of dam building are disclosed in the following section). The effect is more pronounced as water continues to rise, as shown
- The latter dam was built to prevent merging of Water from that basin with water from areas corresponding to the background. This process is continued until the maximum level of flooding (corresponding to the highest gray- level value in the image) is reached.

The final dams correspond to the watershed lines. Which are the desired segmentation results.

- Before proceeding, let us consider how to construct the dams or watershed lines required by watershed segmentation algorithms. Dam construction is based on binary images, which are members of 2-D integer space z^2The simplest way to construct dams separating sets of binary points is to use morphological dilation,